Product Requirements Document (PRD)

AegisAI — Autonomous Disaster Intelligence & Response System

Tagline: "One AI. Every disaster. Every decision." **Document type:** Hackathon PRD / Technical Proposal **Version:** 1.0 **Status:** Draft for build

---

1. Executive Summary

AegisAI is an AI-first disaster response platform where artificial intelligence is the decision-making core of the system, not an add-on feature. The platform ingests multi-source data (satellite, drone, CCTV, weather, seismic, citizen reports, social media), fuses it through a set of specialized AI agents, and outputs explainable, actionable decisions: who to rescue first, which team to send, which route to take, and what will happen next.

Where most disaster dashboards *display* information, AegisAI *decides* — and explains why — while keeping a human approver in the loop for high-stakes actions.

---

2. Problem Statement

- The UN estimates the number of people needing humanitarian assistance has nearly tripled in five years, heading toward 305M+.
- Current disaster response tooling is fragmented: satellite teams, weather teams, call centers, and NGOs often work off separate systems with no unified, real-time, prioritized view.
- Responders are frequently overwhelmed by report volume, unable to separate signal (a real, urgent case) from noise (duplicate, exaggerated, or fake reports).
- Decisions on resource allocation are usually manual, slow, and reactive rather than predictive.

Core insight: The bottleneck is not data collection — it's data fusion, verification, and decisioning under time pressure. That is exactly what AI is suited for.

---

3. Goals & Non-Goals

Goals

- Build an AI decision layer that fuses heterogeneous data sources into prioritized, explainable actions.
- Provide a working real-time dashboard with map visualization (2D primary, 3D for critical zones).
- Demonstrate graceful degradation: the system should keep working (in reduced form) when data sources or connectivity fail.

- Keep a human-in-the-loop for any action that risks life, safety, or major resource commitment.

Non-Goals (be explicit about this in your pitch — judges respect scoping honesty)

- Not attempting to solve global-scale, production-grade national disaster infrastructure in a hackathon timeframe.
- Not claiming 100% accuracy on report verification or predictions — the system exposes confidence scores instead.
- Not replacing human incident commanders — augmenting and accelerating their decisions.

---

4. System Vision & Data Flow



The dashboard is a **visualization and control surface for decisions the AI has already made** — not the thing generating them.

---

5. Feature Set (Detailed)

5.1 Citizen AI Assistant

- **What:** Multi-modal reporting app — citizens submit text, voice, or photo/video reports of distress, damage, or needs.
- **How:** Speech-to-text (Whisper or similar) → NLP entity extraction → image classification for damage severity.
- **Who it saves:** Citizens without access to formal emergency channels; enables reporting in local language and low-literacy contexts via voice.
- **Loophole solved:** Lack of structured, real-time ground-truth data feeding the system.

5.2 Disaster Detection Agent

- **What:** Classifies the type and severity of an unfolding disaster (flood, fire, earthquake, cyclone, landslide) from incoming multi-source data.
- **Inputs:** Satellite imagery, drone video, CCTV, weather feeds, seismic sensor data, citizen reports, social media.
- **Output:** Disaster type, affected zone geometry, severity score, confidence.
- **Loophole solved:** Different disaster types need different detection logic; a single generic model underperforms — this agent routes each event to the correct downstream model.

5.3 Report Verification Agent (Trust Scoring)

- **What:** Assigns every incoming report a confidence/trust score (0–100%) instead of treating all reports as equally credible.
- **Signals used:** GPS consistency, timestamp plausibility, corroborating reports nearby, weather-data consistency, image forensic checks (basic tamper/metadata analysis), satellite/drone confirmation where available, historical pattern matching.
- **Example output:**
  - "Bridge collapsed" — Confidence 97% (14 corroborating reports, drone confirms, rainfall data supports flooding).
  - "Building on fire" — Confidence 28% (single report, no supporting evidence, weather data inconsistent).
- **Loophole solved:** Fake, exaggerated, or duplicate reports wasting rescue resources.

5.4 Victim Detection Agent

- **What:** Estimates number and category of victims (trapped, injured, elderly, children) from drone imagery, thermal data (if available), CCTV, and SOS signals.
- **Loophole solved:** Manual headcounting is slow and error-prone in chaotic environments.

5.5 Route Intelligence Agent

- **What:** Computes and continuously updates the safest, fastest route for rescue teams — not standard road-optimal routing, but disaster-aware routing.
- **Inputs:** Live road-block reports, flood-depth estimates, bridge risk scores, drone imagery, traffic data.
- **Loophole solved:** Standard maps (Google Maps-style) don't account for disaster-specific hazards like flooded or collapsed roads; this agent recomputes routes as new information arrives.

5.6 Resource Commander (Decision Engine core)

- **What:** Converts prioritized needs into concrete, actionable instructions rather than raw availability numbers.
- **Example output:** "Send Team Bravo. Carry 70 food kits. Take Route C. ETA 17 min. Shelter B is full — redirect to Shelter C."
- **Loophole solved:** Most systems show *what's available*; almost none tell responders *what to do with it*. This is the most differentiating feature of the platform.

5.7 Prediction Engine

- **What:** Forecasts how the disaster will evolve — flood spread, fire spread, aftershock likelihood, shelter overcrowding, disease outbreak risk — using current data plus historical disaster patterns.
- **Interactive "what-if" simulation:** A commander can ask, "What happens if rainfall increases by 30%?" and the system projects new flood zones, road closures, and resource needs.
- **Loophole solved:** Reactive-only systems lose valuable response time; prediction lets teams pre-position resources.

5.8 Human Behaviour AI

- **What:** Estimates crowd movement, panic zones, and stampede/evacuation risk from CCTV/drone feeds and population density data.
- **Loophole solved:** Physical hazards aren't only environmental — crowd dynamics themselves cause casualties.

5.9 Volunteer & NGO Coordinator

- **What:** Matches volunteer skills and location to the nearest, most relevant mission automatically (e.g., a nurse near a flooded zone with injured victims gets routed there first).
- **Loophole solved:** Volunteer coordination is usually manual, slow, and inefficient.

5.10 Supply Chain Agent

- **What:** Predicts shortages (food, medicine, fuel, water) *before* they occur, based on consumption rate and incoming demand signals.
- **Loophole solved:** Shortages are typically discovered only after they happen.

5.11 Explainable AI (XAI) Layer

- **What:** Every priority score, allocation, or recommendation is accompanied by a human-readable justification.
- **Example:**

  > Zone A — Priority Score 94 • 52 distress reports • Flood depth estimated at 1.8m • 1 hospital inaccessible • 37 elderly citizens detected • Heavy rainfall expected in 2 hours • Confidence: 97% Recommended: 3 rescue boats, 2 ambulances, 15 medical kits

- **Loophole solved:** Responders won't trust or act on black-box AI output; transparency builds adoption.

5.12 AI Command Center (Conversational Interface)

- **What:** A natural-language interface where a commander can ask questions ("Where should we send rescue teams first?", "What if the river rises another meter?") and receive structured, explainable answers plus simulated outcomes.
- **Loophole solved:** Reduces the need to manually navigate dashboards during high-stress moments.

5.13 Emergency Dashboard (2D + 3D Map)

- **What:** The visualization layer — real-time map showing damage heatmaps, resource positions, active routes, shelter capacity, and confidence-scored incidents.
- **3D approach:** Full photogrammetric 3D reconstruction is computationally expensive and unnecessary for an MVP. Recommended approach:
  - **Base layer:** 2D GIS map (Mapbox GL JS or Deck.gl) with AI overlays — heatmaps, icons, routes.
  - **3D layer (for critical/zoomed zones only):** Use Cesium.js or Mapbox GL's 3D terrain + building extrusion for a "digital twin" view of the most critical zone, optionally enhanced with drone photogrammetry (e.g., via Agisoft/OpenDroneMap-generated meshes) for the MVP's flagship demo zone.
  - This "2D for scale, 3D for focus" approach is realistic within a 48-hour build and still delivers a visually impressive demo.

---

6. Loopholes & Solutions (Consolidated)

#	Loophole	Problem	Solution
1	No internet after disaster	Cell/fiber networks fail, cloud AI unreachable	Edge AI on local devices; mesh networking (LoRa/Wi-Fi Direct); offline-first sync when connectivity returns
2	Satellite images delayed	Overpasses take hours/days	Fuse drone, CCTV, IoT, and crowdsourced data as interim sources
3	Clouds block satellite optical imagery	Optical sensors can't see through cloud/smoke	Use SAR (Synthetic Aperture Radar, e.g. Sentinel-1) which works day/night through cloud cover
4	CCTV unavailable in most zones	Rural/disaster areas often lack coverage	Treat CCTV as optional; prioritize drones + citizen photos/video
5	Every disaster type is different	One-size-fits-all models underperform	Modular AI: disaster-specific models selected by a central orchestrator
6	Real-time 3D mapping is expensive	Full 3D reconstruction is slow and costly	2D GIS + AI overlays for the whole area; 3D only for the flagship critical zone
7	Fake/misleading reports	Wastes rescue resources	Trust/confidence scoring via cross-referencing multiple signals
8	Inaccurate or missing GPS	Many reports lack precise location	NLP-based place-name extraction + geocoding + nearby-report correlation
9	AI misclassification	Wrong predictions send teams to wrong places	Human-in-the-loop approval for all high-risk/critical actions
10	High computation cost	Processing large-scale imagery/text in real time is expensive	Lightweight pretrained models; inference scoped only to affected regions
11	Data privacy	Victim data (photos, phone numbers, location) is sensitive	Encryption, anonymization, role-based access, auto-deletion post-emergency
12	Fragmented regional systems	Different countries/agencies use different formats	Standards-based API layer for GIS, emergency DB, and weather integration

#	Loophole	Problem	Solution
13	No data in low-infrastructure regions	Sparse sensor/digital coverage	Hierarchical fallback: satellite → drone → crowdsourced → first-responder input, degrading gracefully
14	Dynamic/blocked roads	Roads become impassable without warning	Continuous re-routing using live drone/crowdsourced updates
15	Power failure	Devices lose power in the field	Low-power/offline device support, solar chargers, portable batteries

Framing note for judges: No system can eliminate every failure mode — networks fail, sensors break, AI makes mistakes. The credible pitch is not "we solved everything" but "we designed for graceful degradation, transparency, and human oversight at every failure point." This tends to land better with technical judges than claims of completeness.

---

7. Tech Stack

Frontend

- **Framework:** React + TypeScript
- **Mapping:** Mapbox GL JS or Deck.gl (2D + heatmaps), Cesium.js (3D terrain/building view for critical zones)
- **State/data:** React Query, Zustand/Redux
- **Charts:** Recharts / D3.js for prediction visualizations
- **Realtime updates:** WebSockets (Socket.IO)

Backend

- **API layer:** Node.js + Express, or FastAPI (Python) if AI-heavy endpoints dominate
- **Realtime layer:** Socket.IO / WebSocket server for live dashboard updates
- **Task orchestration:** A lightweight agent orchestrator (custom, or a framework like LangGraph) to route data to the correct specialized AI agent
- **Database:** PostgreSQL + PostGIS (geospatial queries) for structured data; Redis for caching/real-time state
- **Message queue:** Kafka or RabbitMQ for ingesting high-volume report/sensor streams (nice-to-have; can be simplified for MVP)

AI / ML Layer

- **NLP (report parsing, entity/location extraction):** spaCy, HuggingFace Transformers, or an LLM API (e.g., Claude/GPT) for structured extraction from free text and voice transcripts

- **Speech-to-text:** Whisper (open-source) or a cloud STT API
- **Image/damage classification:** Pretrained CNN (ResNet/EfficientNet) fine-tuned on disaster-damage datasets, or a vision-language model for zero-shot classification
- **Object/victim detection in imagery:** YOLOv8 for drone/CCTV footage
- **Fake report / trust scoring:** A rules + ML hybrid — a gradient-boosted classifier (XGBoost) combining corroboration count, metadata checks, and geospatial/weather consistency
- **Prediction models:** Simple physics-informed or statistical models for flood/fire spread as MVP stand-ins for full simulation models (e.g., cellular-automaton-based fire spread, simplified hydrological flood models)
- **LLM orchestration for the Command Center:** An LLM (Claude/GPT via API) with function-calling/tool-use to query the decision engine and return structured, explainable answers in natural language

Infrastructure

- **Hosting:** Vercel/Netlify (frontend), Render/Railway/AWS (backend) — pick based on hackathon judging environment
- **Edge/offline:** Service workers + IndexedDB for offline-first citizen app; local inference via ONNX Runtime or TensorFlow Lite for edge devices where feasible to demo
- **Containerization:** Docker for consistent deployment of the agent services

---

8. APIs & Data Sources

Category	Source	Use
Satellite optical imagery	NASA Earthdata / Sentinel Hub (Sentinel-2)	Damage assessment, flood mapping
Satellite radar imagery	Sentinel-1 (SAR, ESA Copernicus)	All-weather/cloud-penetrating imaging
Weather	OpenWeatherMap API, NOAA	Rainfall prediction, storm tracking
Seismic	USGS Earthquake API	Earthquake magnitude/location feeds
Maps/GIS	OpenStreetMap, Mapbox API	Base map, road network, geocoding
Social media	Twitter/X API (where accessible), public disaster hashtag streams	Crowd-sourced situational reports
Historical disaster data	EM-DAT, ReliefWeb, UNOSAT	Model training / pattern baselines for prediction
Flood-specific	NASA Global Flood Monitoring	Flood extent validation
Drone imagery (self-collected/simulated for demo)	Custom capture or simulated dataset	Victim detection, 3D reconstruction of flagship zone

Datasets for model training/fine-tuning (examples to reference in the report):

- xBD (building damage assessment from satellite imagery)
- FloodNet (UAV imagery for flood damage)
- CrisisMMD (multimodal Twitter data for crisis classification)
- AIDER (Aerial Image Dataset for Emergency Response)

9. Architecture Summary

```
[Data Sources] → [Ingestion API Layer] → [Agent Orchestrator]
      → [Specialized AI Agents: Detection / Verification / Victim /
         Route / Resource / Prediction / Behaviour / Volunteer / Supply]
      → [Decision Engine + XAI Layer]
      → [PostgreSQL/PostGIS + Redis]
      → [WebSocket layer] → [React Dashboard: 2D/3D map, alerts, command
center]
      → [Human approval] → [Action dispatched to rescue teams/NGOs]
```

---

10. Human-in-the-Loop & Ethics

- All **critical, resource-committing, or life-safety actions require human approval** before execution — AI recommends, humans confirm.
- Every report/prediction carries a **visible confidence score**; nothing is presented as certain.
- **Privacy:** personal data (names, phone numbers, exact addresses tied to individuals) is encrypted at rest and in transit, access is role-based, and non-essential personal data is purged after the emergency window closes.
- **Bias awareness:** detection/verification models should be evaluated for performance gaps across regions/languages, since disaster data is often skewed toward well-documented areas.

---

11. Success Metrics

- **Response time reduction:** time from report received → resource dispatched, vs. baseline manual process
- **Report triage accuracy:** % of high-priority zones correctly identified in simulated scenarios
- **False positive rate:** fake/low-value reports correctly down-weighted
- **Coverage under degraded conditions:** system still functions (in reduced form) when connectivity/data sources are cut, demoed live
- **User trust/explainability score:** qualitative feedback from judges/testers on whether the reasoning shown is understandable and actionable

---

12. Build Phases

Phase 0 — Setup (Hours 0–4)

- Repo scaffolding, tech stack setup, data source access (API keys), sample dataset collection.

Phase 1 — Core Data Pipeline (Hours 4–14)

- Citizen report ingestion (text/voice/photo), basic NLP location extraction, database schema (PostGIS), base 2D map rendering.

Phase 2 — AI Agents MVP (Hours 14–28)

- Disaster Detection Agent, Report Verification (trust scoring), basic Victim Detection (image-based), Decision Engine v1 (priority scoring + XAI text output).

Phase 3 — Decisioning & Routing (Hours 28–36)

- Resource Commander logic, Route Intelligence (basic re-routing on blocked-road input), Prediction Engine v1 (simple flood/fire spread simulation), what-if simulation UI.

Phase 4 — Dashboard & 3D (Hours 36–42)

- Full dashboard integration, heatmaps, live WebSocket updates, 3D view for one flagship critical zone (Cesium/Mapbox 3D terrain + drone-derived mesh if available, or a stylized 3D approximation if not).

Phase 5 — Command Center & Polish (Hours 42–46)

- Conversational AI Command Center (LLM + function calling into the Decision Engine), UI polish, offline-mode demo path.

Phase 6 — Demo Prep (Hours 46–48)

- Scripted simulated disaster scenario (e.g., Vadodara flood), rehearsal of the full report → detection → verification → decision → dispatch flow, backup recorded demo in case of live-connectivity issues during judging.

---

13. Demo Script (Suggested)

1. A simulated citizen uploads a photo of a flooded street and a voice message: "Five people trapped near XYZ School."
2. Weather API shows heavy rainfall expected in the next 2 hours.
3. System extracts location, classifies flood severity from the image, assigns a trust score to the report.
4. Decision Engine assigns a priority score to the zone with full XAI reasoning shown on screen.
5. Resource Commander recommends a specific team, route, and ETA.
6. Commander asks the AI Command Center: "What if rainfall increases 30%?" — system shows projected new flood zones and shelter overflow.
7. Human operator approves the dispatch; dashboard updates live with the rescue team en route on the 3D map for the flagship zone.

---

14. Risks

Risk	Mitigation
Erroneous AI alerts misdirect rescue teams	Confidence scores + human approval gate for critical actions
Reliance on internet/cloud APIs during demo	Prepare an offline/local-fallback demo path and recorded backup
Real satellite/drone data unavailable for live demo	Use public historical disaster datasets to simulate the live scenario convincingly
Overscoping for a 48-hour hackathon	Prioritize Phases 1–4 as the "must-have" MVP; treat Command Center and full 3D as stretch goals
Privacy concerns with real citizen data	Use synthetic/anonymized data for the demo, never real personal data

---

15. Team Roles (Suggested Split)

- **AI/ML engineer(s):** Detection, verification, prediction models
- **Backend engineer:** API layer, orchestrator, database, WebSocket layer
- **Frontend engineer:** Dashboard, 2D/3D map integration
- **Data/GIS specialist:** Dataset sourcing, geospatial pipeline, PostGIS
- **Product/Pitch lead:** Demo script, slides, framing the "graceful degradation" narrative for judges

---

16. Closing Note for the Pitch

The single strongest differentiator to emphasize to judges is the shift from **"AI as a data-labeling add-on"** to **"AI as the decision-maker, with humans approving high-stakes actions and every recommendation explained."** Most competing hackathon disaster projects stop at a dashboard with a heatmap. AegisAI's edge is the Decision Engine + Resource Commander + Explainable AI layer, plus honest handling of failure modes (offline operation, confidence scoring, graceful degradation) rather than a claim that every loophole is fully closed.